Acceso a Datos · DAM

Novedades de persistencia

Tipos compuestos, arrays y lógica del gestor
Ejemplos nuevos: sensores de invernadero

Docente: José Manuel Villar Ferradal

Aprendizajes nuevos

01 Datos estructurados y colecciones en PostgreSQL

02 Funciones y procedimientos mediante JDBC

03 Documentos JSONB y consultas indexadas

El ejemplo: una red de sensores

01 Cada sensor tiene un nombre y una posición

02 La posición agrupa latitud y longitud

03 Las etiquetas identifican usos o zonas

04 La configuración varía entre dispositivos

Tipos compuestos definidos por el usuario

CREATE TYPE describe una estructura

Cada atributo mantiene su tipo SQL

El tipo puede usarse en una columna

```
CREATE SCHEMA agro;

CREATE TYPE agro.punto_tipo AS (
  latitud numeric(8,5),
  longitud numeric(8,5)
);

SELECT ROW(42.24060, -8.72070)
  ::agro.punto_tipo;
```

Una columna puede contener una estructura

La ubicación guarda un punto completo

text[] representa la colección

JSONB conserva propiedades variables

```
CREATE TABLE agro.sensor (
  id bigint GENERATED ALWAYS AS IDENTITY
    PRIMARY KEY,
  nombre text NOT NULL,
  ubicacion agro.punto_tipo NOT NULL,
  etiquetas text[] NOT NULL
    DEFAULT ARRAY[]::text[],
  config jsonb NOT NULL DEFAULT '{}'::jsonb,
  CHECK (jsonb_typeof(config) = 'object')
);
```

Construcción de compuestos con ROW

ROW agrupa los atributos del punto

El cast identifica el tipo compuesto

Los datos mantienen su estructura

```
INSERT INTO agro.sensor
  (nombre, ubicacion, etiquetas, config)
VALUES (
  'Humedad norte',
  ROW(42.24060, -8.72070)::agro.punto_tipo,
  ARRAY['humedad', 'zona-norte'],
  '{"unidad":"%","intervalo_seg":30}'::jsonb
)
RETURNING id;
```

Proyección de atributos y mapeo Java

El SQL extrae cada atributo interno

Los alias nombran las columnas del resultado

El DAO reconstruye el objeto Java

```
SELECT id, nombre,
       (ubicacion).latitud AS lat,
       (ubicacion).longitud AS lon
FROM agro.sensor;

// Java: dentro de while (rs.next())
Punto punto = new Punto(
  rs.getBigDecimal("lat"),
  rs.getBigDecimal("lon"));
```

Records y composición de objetos

Punto agrupa valores relacionados

Sensor contiene otro objeto

Los accesores se llaman latitud() o nombre()

```
public record Punto(
    BigDecimal latitud,
    BigDecimal longitud) { }

public record Sensor(
    long id, String nombre, Punto ubicacion,
    List<String> etiquetas,
    String configJson) { }
```

Arrays SQL y consultas sobre colecciones

ANY comprueba pertenencia

cardinality cuenta los elementos

unnest transforma elementos en filas

```
SELECT nombre FROM agro.sensor
WHERE 'humedad' = ANY(etiquetas);

SELECT nombre, cardinality(etiquetas)
FROM agro.sensor;

SELECT s.nombre, e.etiqueta
FROM agro.sensor s
LEFT JOIN LATERAL unnest(s.etiquetas)
  AS e(etiqueta) ON true;
```

Envío de arrays con JDBC

createArrayOf indica el tipo SQL

setArray envía la colección

free libera el contenedor JDBC

```
String sql = """
  UPDATE agro.sensor SET etiquetas = ?
  WHERE id = ?
  """;
try (var ps = c.prepareStatement(sql)) {
  java.sql.Array a = c.createArrayOf(
    "text", new String[]{"riego", "prioritario"});
  try {
    ps.setArray(1, a);
    ps.setLong(2, idSensor);
    ps.executeUpdate();
  } finally { a.free(); }
}
```

Lectura de arrays y conversión a List

getArray recupera el contenedor SQL

Los elementos se copian a una lista

Array utiliza free en lugar de close

```
java.sql.Array a = rs.getArray("etiquetas");
List<String> etiquetas;
try {
  String[] valores = (String[]) a.getArray();
  etiquetas = new ArrayList<>(
    Arrays.asList(valores));
} finally {
  a.free();
}
```

Lógica que ejecuta el gestor

01 El procedimiento realiza una operación

02 La función devuelve un resultado tipado

03 CallableStatement conecta Java con las rutinas

04 Las validaciones pueden compartirse entre aplicaciones

Procedimiento: cambio de latitud

Los parámetros reciben id y nueva coordenada

El gestor valida el intervalo permitido

FOUND detecta identificadores ausentes

```
CREATE PROCEDURE agro.cambiar_latitud(
  IN p_id bigint, IN p_lat numeric)
LANGUAGE plpgsql AS $$
BEGIN
  IF p_lat IS NULL OR abs(p_lat) > 90 THEN
    RAISE EXCEPTION 'Latitud fuera de rango';
  END IF;
  UPDATE agro.sensor
  SET ubicacion.latitud = p_lat WHERE id = p_id;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'Sensor inexistente';
  END IF;
END;
$$;
```

Invocación de un procedimiento desde Java

prepareCall prepara la llamada

setBigDecimal corresponde a numeric

El driver debe generar CALL sin retorno

```
try (var cs = c.prepareCall(
    "{call agro.cambiar_latitud(?, ?)}")) {
  cs.setLong(1, idSensor);
  cs.setBigDecimal(2,
    new BigDecimal("42.25000"));
  cs.execute();
}

// Opción al abrir la conexión:
opciones.setProperty("escapeSyntaxCallMode",
                     "callIfNoReturn");
```

Función: sensores con una etiqueta

RETURNS declara el valor devuelto

La consulta reutiliza ANY sobre el array

La función también puede llamarse con SELECT

```
CREATE FUNCTION agro.sensores_con_etiqueta(
  p_etiqueta text)
RETURNS bigint
LANGUAGE sql STABLE AS $$
  SELECT count(*)
  FROM agro.sensor
  WHERE p_etiqueta = ANY(etiquetas)
$$;

SELECT agro.sensores_con_etiqueta('humedad');
```

Parámetros de salida y retorno JDBC

El primer ? recibe el resultado

El segundo ? aporta la entrada

El tipo se registra antes de ejecutar

```
try (var cs = c.prepareCall(
    "{? = call agro.sensores_con_etiqueta(?)}")) {
  cs.registerOutParameter(1, Types.BIGINT);
  cs.setString(2, "humedad");
  cs.execute();
  long total = cs.getLong(1);
}
```

JSONB para configuraciones variables

Cada dispositivo puede añadir propiedades

Las claves pueden contener listas u objetos

JSONB permite consultar el contenido

```
{
  "unidad": "%",
  "intervalo_seg": 30,
  "calibracion": {
    "offset": 0.8,
    "fecha": "2026-10-20"
  },
  "alarmas": ["bateria", "sin_senal"]
}
```

JSON válido y reglas de negocio

El tipo comprueba la sintaxis JSON

CHECK puede exigir claves y valores

El valor por defecto debe cumplir las reglas

```
ALTER TABLE agro.sensor
ALTER COLUMN config
SET DEFAULT '{"unidad":"C"}'::jsonb;

ALTER TABLE agro.sensor ADD CHECK (
  config ? 'unidad'
  AND jsonb_typeof(config->'unidad') = 'string'
  AND config->>'unidad' IN ('C', '%')
);
```

Escritura parametrizada de JSONB

El documento viaja como texto

El cast solicita la conversión a JSONB

El UPDATE reemplaza toda la configuración

```
String sql = """
  UPDATE agro.sensor SET config = ?::jsonb
  WHERE id = ?
  """;
try (var ps = c.prepareStatement(sql)) {
  ps.setString(1,
    "{\"unidad\":\"%\",\"intervalo_seg\":60}");
  ps.setLong(2, idSensor);
  ps.executeUpdate();
}
```

Consultas sobre el documento

->> extrae un valor como texto

@> comprueba contención

? comprueba si existe una clave

```
SELECT nombre, config->>'unidad' AS unidad
FROM agro.sensor;

SELECT nombre FROM agro.sensor
WHERE config @> '{"unidad":"%"}'::jsonb;

SELECT nombre FROM agro.sensor
WHERE config ? 'calibracion';
```

Índice GIN y planes de ejecución

GIN puede apoyar la búsqueda por contención

EXPLAIN muestra el plan elegido

El tamaño y la selectividad influyen

```
CREATE INDEX sensor_config_gin
ON agro.sensor USING gin(config);

EXPLAIN SELECT *
FROM agro.sensor
WHERE config @> '{"unidad":"%"}'::jsonb;
```

TableView, records y selección

Cada fila conserva un objeto Sensor

La columna usa su accesor nombre()

El listener copia el objeto al formulario

```
colNombre.setCellValueFactory(fila ->
  new ReadOnlyStringWrapper(
    fila.getValue().nombre()));

tabla.getSelectionModel()
  .selectedItemProperty()
  .addListener((obs, anterior, sensor) -> {
    if (sensor != null) {
      txtNombre.setText(sensor.nombre());
    }
  });
```

Task para consultas sin bloquear la ventana

call ejecuta el trabajo en segundo plano

onSucceeded actualiza los controles

onFailed permite presentar el error

```
Task<List<Sensor>> tarea = new Task<>() {
  @Override protected List<Sensor> call()
      throws SQLException {
    return repositorio.listar();
  }
};
tarea.setOnSucceeded(e ->
  tabla.getItems().setAll(tarea.getValue()));
tarea.setOnFailed(e ->
  mostrarError(tarea.getException()));
ejecutor.execute(tarea);
```

Validación y diagnóstico de errores

01 La interfaz detecta entradas incompletas

02 La base protege reglas desde cualquier acceso

03 SQLState permite identificar errores

04 Los avisos deben permitir continuar trabajando

Portabilidad y elección de estructuras

01 pgJDBC 42.7.8 no implementa createStruct

02 ROW y proyección resuelven el compuesto

03 Una etiqueta con atributos puede necesitar tabla

04 Las reglas estables ayudan a decidir qué va en JSONB